LAB 7 – FUZZ TESTING FOR INPUT VALIDATION BUGS

Experiment: Fuzz Testing for Input Validation Bugs

Name: Praakruthi PS

SRN: PES1UG24CS324

---

1. OBJECTIVE

To explore fuzz testing as a security and quality assurance technique by identifying and fixing input validation bugs in a Python application using the Hypothesis fuzz testing library.

The experiment focuses on generating unexpected and invalid inputs, identifying potential failures, and improving application robustness through defensive programming.

2. APPLICATION OVERVIEW

The Parking Fee application consists of three main functions:

1. minutes_parked():

Calculates the parking duration in minutes based on the vehicle's entry and exit times.

2. hourly_rate():

Returns the hourly parking rate based on the vehicle type (bike, car, or truck).

3. format_plate():

Formats a vehicle registration number into the required structure.

The original application was executed with valid inputs to observe its expected behaviour before performing fuzz testing.

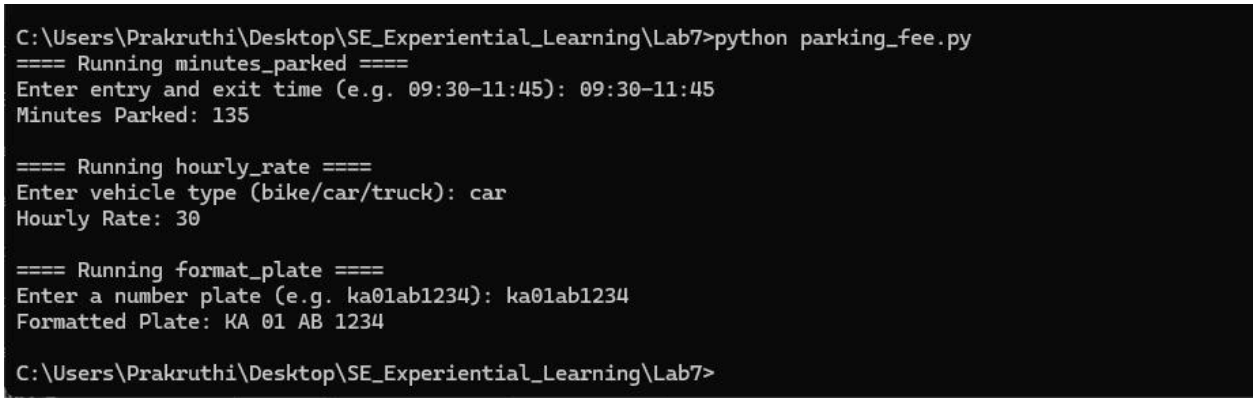


Figure 1: Execution of the original Parking Fee application with valid inputs.

---

4. FUZZ TESTING USING HYPOTHESIS

Hypothesis is a property-based testing library for Python that automatically generates a variety of inputs to identify unexpected behaviour and potential bugs.

Three fuzz tests were implemented using the `@given` decorator and Hypothesis strategies.

The following strategies were used:

- `st.text()` – Generates random Unicode strings.
- `st.none()` – Generates None values.

The three functions were tested independently to identify failures caused by unexpected or malformed inputs.

4.1 TEST EXECUTION ON THE ORIGINAL APPLICATION

The test suite was executed using the following command:

```
python -m pytest test_parking_fee.py -v
```

Hypothesis generated different inputs and exposed failures in the original implementation.

The initial test execution resulted in:

- 4 failed tests
- 3 passed tests

The failures revealed that the original application was not sufficiently equipped to handle unexpected inputs.

SCREENSHOT 2 – FAILING TESTS

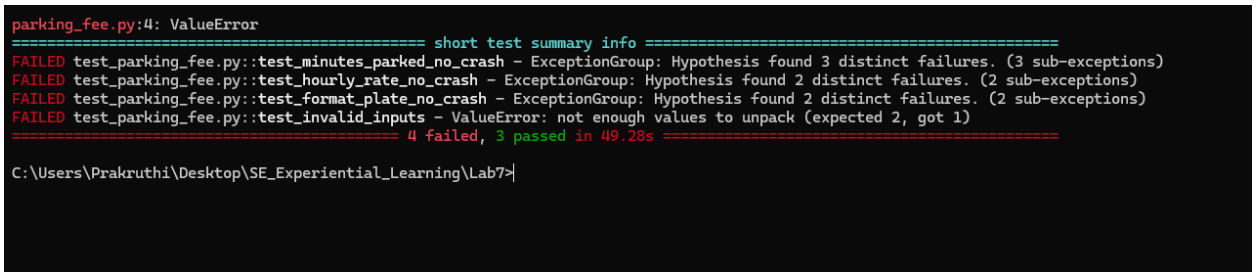


Figure 2: Hypothesis fuzz testing results showing failures in the original application.

4.2 BUGS IDENTIFIED

The following input validation issues were identified:

Function	Identified Issue
minutes_parked()	Invalid time formats and unexpected inputs caused exceptions.
hourly_rate()	Unknown vehicle types caused lookup errors.
format_plate()	Invalid or short registration numbers caused errors.

These failures demonstrated the importance of testing applications with unexpected and malformed inputs rather than relying only on normal test cases.

---

5. BUG FIXING AND DEFENSIVE PROGRAMMING

The identified issues were addressed by modifying the original `parking_fee.py` application.

The following improvements were implemented:

1. Input Type Validation

Used `isinstance()` to verify that inputs are strings before processing them.

2. Time Format Validation

Used regular expressions to validate the entry and exit time format and ensure that hours and minutes fall within valid ranges.

3. Safe Dictionary Lookup

Used the dictionary `get()` method in `hourly_rate()` to handle unsupported vehicle types safely.

4. Vehicle Registration Validation

Validated the registration number format before performing string slicing and formatting.

5. Invalid Input Handling

Modified the functions to return `None` for invalid inputs instead of raising unexpected exceptions.

These changes improved the application's robustness and ensured predictable handling of invalid inputs.

5.1 EXECUTION OF THE FIXED APPLICATION

The modified application was executed again using:

```
python parking_fee.py
```

The application successfully processed valid inputs and produced the expected results.

SCREENSHOT 3 – FIXED APPLICATION OUTPUT

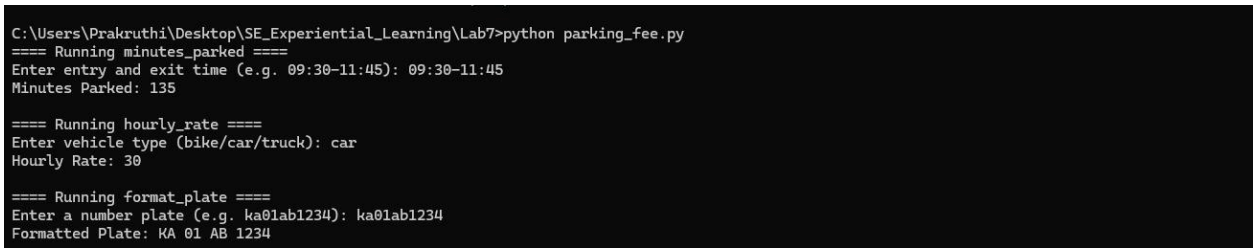


Figure 3: Execution of the modified Parking Fee application after implementing input validation and defensive programming.

---

6. FINAL TESTING AND VERIFICATION

After fixing the application, the same test suite was executed again using:

```
python -m pytest test_parking_fee.py -v
```

The tests were performed against the modified application to verify that the identified issues had been addressed.

The final test suite included:

- Three Hypothesis fuzz tests covering all three application functions.
- Three functional tests verifying expected outputs for valid inputs.
- One additional test verifying the handling of invalid inputs.

All seven tests passed successfully.

SCREENSHOT 4 – FINAL TEST RESULTS



Figure 4: Final pytest execution showing all seven tests passing successfully.

---

7. RESULT AND CONCLUSION

Fuzz testing was successfully performed on the Parking Fee application using the Hypothesis library.

Random and unexpected inputs were generated to identify input validation bugs in the three application functions: `minutes_parked()`, `hourly_rate()`, and `format_plate()`.

The identified issues were addressed using defensive programming techniques, input type checking, regular expressions, and safe dictionary lookups.

After implementing the fixes, all seven tests passed successfully, demonstrating improved robustness against invalid and unexpected inputs. **Thus, the experiment demonstrated the effectiveness of property-based fuzz testing in identifying edge-case bugs and improving the reliability of Python applications.**